

Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Сибирский государственный университет телекоммуникаций и
информатики»
(СибГУТИ)

Кафедра ТСиВС

ОТЧЕТ ПО РАСЧЕТНО-ГРАФИЧЕСКОЙ РАБОТЕ

**По дисциплине: «Интернет-технологии и сетевое
программное обеспечение»**

**Тема: «Разработка микросервисного приложения likes-s18 с
интеграцией REST и gRPC»**

Студент группы ИКС-433
ФИО студента Петров Роман Ю.

Проверил:
Должность преподаватель
ФИО Лошкарев А.В.

Новосибирск 2026 г.

Оглавление

Введение	3
1 Реализация микросервисного проекта РГР	5
1.1 Постановка задачи	5
1.2 Архитектура системы	5
1.3 Выбор протоколов взаимодействия	6
1.4 REST API сервисов	7
1.5 gRPC-контракт и межсервисный вызов	7
1.6 Запуск REST и gRPC внутри like-service	9
1.7 Отказоустойчивость и healthcheck	9
1.8 Контейнеризация через Docker	10
1.9 Оркестрация через Docker Compose	11
1.10 Настройка CI	12
1.11 Kubernetes-манифесты	13
1.12 Continuous Delivery (CD)	14
1.13 Результаты выполнения РГР	15
1.14 Ответы на вопросы самопроверки	16
Заключение	17
Список использованных источников	18

Введение

В рамках расчетно-графической работы выполнен финальный микросервисный проект `likes-s18`. Проект представляет собой небольшую распределенную систему, состоящую из нескольких независимых сервисов, отдельных баз данных PostgreSQL, REST API для внешних клиентов и gRPC-взаимодействия для внутреннего обмена данными. Такой формат работы позволяет не только написать отдельные HTTP-обработчики, но и показать полный цикл проектирования сервиса: от выделения доменных границ до контейнеризации и автоматической проверки в CI [1, 2].

Актуальность работы определяется практической необходимостью проектировать системы, которые:

- можно запускать и обновлять независимо по сервисам;
- устойчивы к частичным отказам;
- удобно разворачиваются в контейнерной среде.

Цель РГР – спроектировать и реализовать микросервисное приложение, состоящее из `user-service`, `post-service` и `like-service`. Важной частью реализации является сценарий, при котором `post-service` получает количество лайков по gRPC-вызову `GetLikesCount`, а затем возвращает клиенту REST-ответ с дополнительным полем `likes_count`.

Для достижения цели были поставлены и решены следующие задачи:

- выделение доменных границ и разделение функциональности по сервисам;
- разработка REST API для CRUD-операций с пользователями, постами и лайками;
- описание и использование gRPC-контракта на базе `.proto`;
- организация инфраструктуры через Docker Compose и настройка `healthcheck`;
- подготовка Kubernetes-манифестов для разворачивания сервисов и баз данных;
- реализация CI/CD-сценария для тестирования, публикации образов и деплоя.

В отчете приведены принятые архитектурные решения, описание реализации REST и gRPC-части, контейнеризация через Docker и Docker Compose, оркестрация в Kubernetes и настройка CI/CD-проверки проекта [3, 4, 8, 10].

1 Реализация микросервисного проекта РГР

1.1 Постановка задачи

В расчетно-графической работе разработан проект `likes-s18`. Основная идея проекта – разделить функциональность социальной платформы на несколько независимых сервисов и связать их через разные протоколы взаимодействия. В результате получилась система, в которой каждый сервис отвечает за отдельный домен и может развиваться независимо.

В состав проекта входят три сервиса:

- `user-service` – регистрация и получение пользователей;
- `post-service` – создание, получение и удаление постов;
- `like-service` – создание лайков, получение лайков и подсчет их количества.

Ключевой пользовательский сценарий выглядит следующим образом: клиент создает пост, затем создает лайки к этому посту, после чего запрашивает пост по идентификатору. При выдаче поста `post-service` обращается к `like-service` по gRPC и добавляет в ответ поле `likes_count`. Благодаря этому клиенту не нужно выполнять дополнительный запрос для подсчета лайков.

1.2 Архитектура системы

Разделение на сервисы выполнено по бизнес-доменам. Пользователи, посты и лайки имеют разную ответственность, разные модели данных и потенциально разную нагрузку. Поэтому каждый домен вынесен в отдельный сервис и работает со своей базой PostgreSQL [5].

Архитектура проекта построена по следующей схеме:

- 1) внешний клиент обращается к REST API сервисов;

- 2) `post-service` хранит и возвращает данные постов;
- 3) `like-service` хранит данные лайков;
- 4) при получении конкретного поста `post-service` выполняет gRPC-вызов в `like-service`;
- 5) `like-service` подсчитывает лайки в своей базе данных и возвращает результат.

Такое разделение снижает связанность. Например, изменение логики подсчета лайков не требует изменять код сервиса пользователей. Аналогично, если нужно масштабировать обработку лайков, можно увеличить количество экземпляров только `like-service`, не затрагивая остальные части системы [4].

1.3 Выбор протоколов взаимодействия

Для внешнего API выбран REST, так как он прост в ручном тестировании, хорошо поддерживается браузерами, HTTP-клиентами и FastAPI [1]. Для внутреннего взаимодействия между `post-service` и `like-service` выбран gRPC. Он удобен тем, что контракт описывается в `.proto`-файле, а клиентский и серверный код используют сгенерированные классы [2].

Сравнение используемых протоколов приведено в таблице 1.1.

Таблица 1.1 – Сравнение REST и gRPC в проекте

Критерий	REST	gRPC	
Назначение	Внешнее API	Внутренний вызов подсчета лайков	
Формат данных	JSON	Protobuf	
Контракт	HTTP endpoint-ы	<code>likes.proto</code>	
Удобство	Просто проверять через curl	Удобно для сервис-сервис связи	

1.4 REST API сервисов

REST API реализовано на FastAPI. Каждый сервис создает собственное приложение и описывает набор endpoint-ов для работы со своими сущностями. Например, в `like-service` реализованы методы создания лайка, получения списка лайков, получения лайка по идентификатору, удаления лайка и отдельный HTTP endpoint для подсчета лайков по посту.

Фрагмент REST-обработчика создания лайка приведен в листинге 1.1.

Листинг 1.1 – Создание лайка в `like-service`

```
@app.post("/likes")
def create_like(post_id: int, db: Session = Depends(get_db)):
    new_like = Like(post_id=post_id)
    db.add(new_like)
    db.commit()
    db.refresh(new_like)
    return new_like
```

В этом обработчике FastAPI получает параметр `post_id`, открывает сессию SQLAlchemy через зависимость `Depends(get_db)`, создает объект модели `Like`, сохраняет его в базе данных и возвращает созданную запись. Аналогичный подход используется в сервисах пользователей и постов [6].

1.5 gRPC-контракт и межсервисный вызов

gRPC-контракт описывает один метод `GetLikesCount`. На вход метод принимает идентификатор поста, а в ответ возвращает количество лайков. Контракт приведен в листинге 1.2.

Листинг 1.2 – gRPC-контракт `likes.proto`

```
syntax = "proto3";

service LikeService {
    rpc GetLikesCount (LikeRequest) returns (LikeResponse);
}
```

```

message LikeRequest {
    int32 post_id = 1;
}

message LikeResponse {
    int32 count = 1;
}

```

Серверная часть gRPC находится в `like-service`. Она получает `post_id`, выполняет запрос к таблице лайков и возвращает количество найденных записей. Основной фрагмент реализации показан в листинге 1.3.

Листинг 1.3 – Метод `GetLikesCount` на стороне `like-service`

```

def GetLikesCount(self, request, context):
    db = SessionLocal()
    count = db.query(Like).filter(
        Like.post_id == request.post_id
    ).count()
    return likes_pb2.LikeResponse(count=count)

```

На стороне `post-service` создается gRPC-канал до контейнера `like-service`. Затем вызывается метод `GetLikesCount`, а результат добавляется в REST-ответ клиенту. Этот фрагмент приведен в листинге 1.4.

Листинг 1.4 – Вызов gRPC в `post-service`

```

channel = grpc.insecure_channel("like-service:50051")
stub = likes_pb2_grpc.LikeServiceStub(channel)
response = stub.GetLikesCount(
    likes_pb2.LikeRequest(post_id=post_id)
)
likes_count = response.count

```

Важная деталь состоит в том, что имя `like-service` используется как сетевое имя внутри Docker Compose. Благодаря встроенной DNS-сети Compose сервисы могут обращаться друг к другу по именам, указанным в файле `docker-compose.yml`. В Kubernetes используется тот же подход: внутри namespace сервис доступен по DNS-имени `like-service`.

1.6 Запуск REST и gRPC внутри like-service

like-service должен одновременно обслуживать HTTP-запросы FastAPI и gRPC-запросы от post-service. Для этого gRPC-сервер запускается в отдельном потоке во время старта FastAPI-приложения. Фрагмент запуска показан в листинге 1.5.

Листинг 1.5 – Запуск gRPC-сервера в lifespan FastAPI

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    threading.Thread(
        target=start_grpc_server,
        daemon=True
    ).start()
    print("gRPC thread started")
    yield
    print("Application shutdown")
```

Такое решение позволяет поднять оба интерфейса в одном контейнере: REST API работает на порту 8000, а gRPC-сервер слушает порт 50051. При этом внешнему клиенту доступен REST-интерфейс, а внутренние сервисы используют gRPC.

1.7 Отказоустойчивость и healthcheck

В post-service gRPC-вызов обернут в try/except. Если like-service временно недоступен, сервис постов не падает, а возвращает пост с likes_count = 0. Это пример деградации функциональности: клиент все равно получает основную информацию о посте, но без точного количества лайков.

Листинг 1.6 – Обработка ошибки gRPC-вызова

```
try:
    response = stub.GetLikesCount(
        likes_pb2.LikeRequest(post_id=post_id)
    )
```

```

        likes_count = response.count
    except Exception as e:
        print("gRPC ERROR:", e)
        likes_count = 0

```

Для проверки состояния сервисов реализован endpoint `/health`. Он не просто возвращает статический ответ, а выполняет запрос `SELECT 1` к базе данных. Поэтому healthcheck проверяет не только доступность HTTP-приложения, но и подключение к хранилищу.

Листинг 1.7 – Проверка состояния сервиса

```

@app.get("/health")
def health(db: Session = Depends(get_db)):
    try:
        db.execute(text("SELECT 1"))
    except Exception as e:
        raise HTTPException(
            status_code=503,
            detail=f"database is unavailable: {e}"
        )
    return {"status": "ok", "service": "post-service"}

```

1.8 Контейнеризация через Docker

Каждый сервис упакован в отдельный Docker-образ. Dockerfile задает базовый Python-образ, рабочую директорию, установку зависимостей, копирование исходного кода и команду запуска FastAPI-приложения через `uvicorn`. Пример Dockerfile для `post-service` приведен в листинге 1.8.

Листинг 1.8 – Dockerfile сервиса постов

```

FROM python:3.12

WORKDIR /app

COPY ./post_service/requirements.txt ./requirements.txt

RUN pip install --no-cache-dir -r requirements.txt

```

```
COPY ./post_service /app
COPY ./proto /app/proto

CMD ["python", "-m", "uvicorn", "posts:app",
    "--host", "0.0.0.0", "--port", "8000"]
```

Отдельная упаковка сервисов дает несколько преимуществ: окружение становится воспроизводимым, зависимости не смешиваются между сервисами, а запуск приложения не зависит от локальной настройки Python на машине разработчика [7].

1.9 Оркестрация через Docker Compose

Для локального запуска всех компонентов используется Docker Compose. В одном файле описаны три API-сервиса и три базы данных PostgreSQL. Внешние порты проброшены следующим образом:

- like-service: REST – 8001, gRPC – 50051;
- post-service: REST – 8002;
- user-service: REST – 8003.

Фрагмент конфигурации like-service приведен в листинге 1.9.

Листинг 1.9 – Фрагмент docker-compose.yml для like-service

```
like-service:
  build:
    context: .
    dockerfile: like_service/Dockerfile
  ports:
    - "8001:8000"
    - "50051:50051"
  depends_on:
    like-db:
      condition: service_healthy
  healthcheck:
    test: ["CMD-SHELL", "python -c \"import urllib.request;
      urllib.request.urlopen('http://127.0.0.1:8000/health',
      timeout=3).read()\""]
    interval: 10s
    timeout: 5s
```

```
retries: 5
start_period: 20s
```

Параметр `depends_on` с условием `service_healthy` нужен для корректного порядка запуска. API-контейнер стартует только после того, как контейнер базы данных пройдет проверку готовности. Для PostgreSQL используется команда `pg_isready`; она позволяет понять, что СУБД уже принимает подключения.

Запуск всей системы выполняется одной командой из каталога приложения:

Листинг 1.10 – Запуск контейнеров проекта

```
docker compose up -d --build
```

1.10 Настройка CI

Для проекта настроен GitHub Actions workflow, который автоматически проверяет РГР при изменении кода сервисов, вариантов, зависимостей или самого файла CI. Pipeline выполняется в окружении `ubuntu-latest` и использует переменные `GROUP=433` и `STUDENT_ID=s18`, чтобы тесты брали правильный вариант проекта.

Основные шаги CI:

- 1) checkout репозитория;
- 2) установка Python 3.12;
- 3) установка зависимостей из `requirements.txt`;
- 4) запуск тестов командой `make test WEEK=17`;
- 5) сборка и запуск контейнеров через Docker Compose;
- 6) smoke-тест REST + gRPC-сценария;
- 7) вывод логов при ошибке;
- 8) остановка контейнеров и удаление volume-ов.

Фрагмент настройки CI приведен в листинге 1.11.

Листинг 1.11 – Фрагмент GitHub Actions CI

```
- name: Run tests
  run: make test WEEK=17
```

```

- name: Build and start services
  run: docker compose -f "$APP_DIR/docker-compose.yml" up --
      build -d

- name: Smoke test REST + gRPC path
  run: |
    sleep 20
    docker compose -f "$APP_DIR/docker-compose.yml" exec -T
      like-service python -c "import socket; s=socket.socket
      (); s.settimeout(3); assert s.connect_ex
      (('127.0.0.1', 50051)) == 0"
    docker compose -f "$APP_DIR/docker-compose.yml" exec -T
      post-service python -c "import requests; p=requests.
      post('http://127.0.0.1:8000/posts', params={'title': 'ci
      -smoke', 'content': 'ok'}).json(); r=requests.get(f'http
      ://127.0.0.1:8000/posts/{p['id']}\"); assert r.
      status_code == 200; assert 'likes_count' in r.json()"

```

Smoke-тест важен тем, что он проверяет не только наличие файлов или прохождение статического теста, но и фактический запуск системы в контейнерах. Сначала проверяется, что gRPC-порт 50051 открыт внутри like-service. Затем внутри post-service создается пост и выполняется запрос на получение этого поста. Успешная проверка поля likes_count подтверждает, что цепочка post-service -> gRPC -> like-service работает.

1.11 Kubernetes-манифесты

Для кластерного развертывания добавлена директория weeks/week-17/k8s, в которой описаны все ресурсы проекта:

- namespace likes-s18;
- Deployment + Service для user-service, post-service, like-service;
- Deployment + Service для user-db, post-db, like-db;
- файл kustomization.yaml для применения полного набора манифестов.

Фрагмент манифеста like-service приведен в листинге 1.12.

Листинг 1.12 – Фрагмент Kubernetes-манифеста like-service

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: like-service
  namespace: likes-s18
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: like-service
          image: ghcr.io/example/likes-s18-like-service:latest
          ports:
            - containerPort: 8000
            - containerPort: 50051
          readinessProbe:
            httpGet:
              path: /health
              port: 8000
```

В этом манифесте фиксируется желаемое число реплик, образ контейнера, открытые порты и проверка готовности через endpoint `/health`. Наличие `readiness/liveness`-проб позволяет оркестратору не отправлять трафик в неготовые pod-ы и автоматически перезапускать их при сбоях [8].

Базы данных также описаны как отдельные `Deployment/Service`-ресурсы. Для PostgreSQL используется probe на основе `pg_isready`, что обеспечивает корректный запуск API только после готовности хранилища.

1.12 Continuous Delivery (CD)

Отдельный workflow `week-17-cd.yml` реализует конвейер доставки:

- 1) запуск тестов;
- 2) сборка и публикация трех образов в GHCR;
- 3) применение Kubernetes-манифестов;
- 4) обновление образов в Deployment через `kubectl set image`;
- 5) ожидание завершения `rolling update` через `kubectl rollout status`.

Ключевой фрагмент CD приведен в листинге 1.13.

Листинг 1.13 – Шаги deploy в GitHub Actions CD

```
- name: Apply manifests
  run: |
    kubectl apply -f weeks/week-17/k8s/namespace.yaml
    kubectl apply -f weeks/week-17/k8s/

- name: Update images to current commit
  run: |
    kubectl -n "${K8S_NAMESPACE}" set image deployment/like-
      service \
      like-service=ghcr.io/${{ github.repository }}/week-17-
      like-service:${{ github.sha }}

- name: Wait for rollout
  run: kubectl -n "${K8S_NAMESPACE}" rollout status deployment
      /like-service --timeout=180s
```

Таким образом, при изменении кода в ветке main pipeline доводит систему до нового согласованного состояния в кластере: тесты проходят, образы публикуются, а деплой обновляется без полного простоя за счет rolling update [10, 9].

1.13 Результаты выполнения РГР

В результате выполнения РГР получена микросервисная система, которая:

- реализует REST API для пользователей, постов и лайков;
- использует gRPC для внутреннего подсчета лайков;
- хранит данные каждого сервиса в отдельной базе PostgreSQL;
- запускается локально через Docker Compose;
- имеет healthcheck-и для API и баз данных;
- автоматически проверяется в GitHub Actions CI;
- разворачивается в Kubernetes через набор манифестов;
- поддерживает CD-сценарий публикации образов и rolling update.

Главным результатом является не только работоспособность отдельных

endpoint-ов, но и проверенная интеграция нескольких компонентов: HTTP API, gRPC, баз данных, контейнеров и CI-сценария.

1.14 Ответы на вопросы самопроверки

1. Критерии разделения на сервисы. Разделение выполнено по бизнес-возможностям системы: пользователи, посты, лайки. Каждый сервис владеет своим API и своей базой данных. Такая модель уменьшает связанность модулей и позволяет масштабировать наиболее нагруженные части системы отдельно.

2. Причины выбора стека. Python и FastAPI выбраны за высокую скорость разработки и встроенную поддержку OpenAPI. PostgreSQL использован как стабильная реляционная СУБД. REST применен для клиентского API, а gRPC – для внутреннего вызова с явным контрактом через `.proto`.

3. Обработка ошибок и устойчивость к сбоям. Критичный внешний endpoint сервиса постов реализует безопасное поведение при ошибке gRPC-вызова: исключение перехватывается, сервис не падает, а клиент получает валидный ответ с нулевым значением лайков.

4. Подход к обновлению без простоя. В проект добавлен CD workflow для Kubernetes: после тестов публикуются новые образы, затем выполняется `kubectl set image` и контролируется `kubectl rollout status`. За счет `readinessProbe` и механизма `rolling update` трафик переключается на новые pod-ы постепенно, что позволяет обновлять сервисы без полной остановки системы.

5. Наиболее сложный этап интеграции. Сложность представлял одновременный запуск REST и gRPC в `like-service`. Для корректной работы gRPC-сервер был вынесен в отдельный поток и запущен в `lifecycle` FastAPI-приложения.

6. Возможные улучшения. При дальнейшем развитии проекта рекомендуется добавить централизованное логирование, сбор метрик Prometheus/Grafana, `retry/backoff` и `circuit breaker` для внутренних вызовов, а также API Gateway для маршрутизации и единой точки авторизации.

Заключение

В ходе выполнения РГР разработан и интегрирован микросервисный проект `likes-s18`, включающий три изолированных сервиса, отдельные базы данных и межсервисное взаимодействие по gRPC.

Поставленная цель достигнута: реализован сценарий получения поста с динамическим подсчетом лайков через внутренний вызов `GetLikesCount`. Дополнительно обеспечены контейнеризация, `healthcheck` и автоматизированная проверка ключевого сценария в CI. `Dockerfile`-ы описывают сборку отдельных сервисов, `Docker Compose` объединяет API и базы данных в одно локальное окружение, `Kubernetes` манифесты задают целевое состояние системы в кластере, а `GitHub Actions` реализует CI/CD-цепочку для тестов, публикации образов и деплоя.

Практический результат работы заключается в формировании целостного прототипа распределенной системы, где учтены базовые требования к связности, надежности и удобству сопровождения. Возможные направления развития: внедрение централизованного логирования и метрик, добавление `retry` и `circuit breaker`, а также переход от базовых манифестов к `Helm`-чарту и секретам для конфигурации `production`-среды.

Литература

- [1] FastAPI Documentation [Электронный ресурс]. URL: <https://fastapi.tiangolo.com/> (дата обращения: 27.05.2026).
- [2] gRPC Documentation [Электронный ресурс]. URL: <https://grpc.io/docs/> (дата обращения: 27.05.2026).
- [3] Материалы РГР: финальный микросервисный проект. Локальный файл проекта README.md (дата обращения: 27.05.2026).
- [4] Архитектура финального проекта РГР. Локальный файл проекта ARCHITECTURE.md (дата обращения: 27.05.2026).
- [5] PostgreSQL Documentation [Электронный ресурс]. URL: <https://www.postgresql.org/docs/> (дата обращения: 27.05.2026).
- [6] SQLAlchemy Documentation [Электронный ресурс]. URL: <https://docs.sqlalchemy.org/> (дата обращения: 27.05.2026).
- [7] Docker Compose Documentation [Электронный ресурс]. URL: <https://docs.docker.com/compose/> (дата обращения: 27.05.2026).
- [8] Kubernetes Documentation [Электронный ресурс]. URL: <https://kubernetes.io/docs/> (дата обращения: 27.05.2026).
- [9] kubectl Reference [Электронный ресурс]. URL: <https://kubernetes.io/docs/reference/kubectl/> (дата обращения: 27.05.2026).
- [10] GitHub Actions Documentation [Электронный ресурс]. URL: <https://docs.github.com/actions> (дата обращения: 27.05.2026).